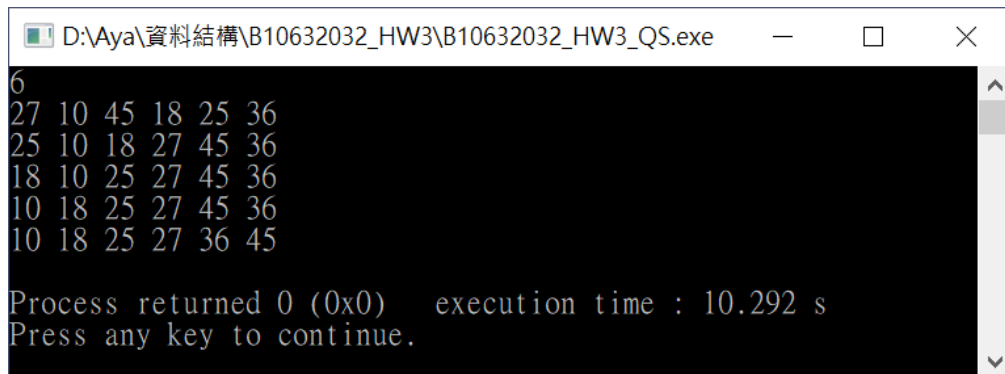


Sort Algorithm

一、執行畫面

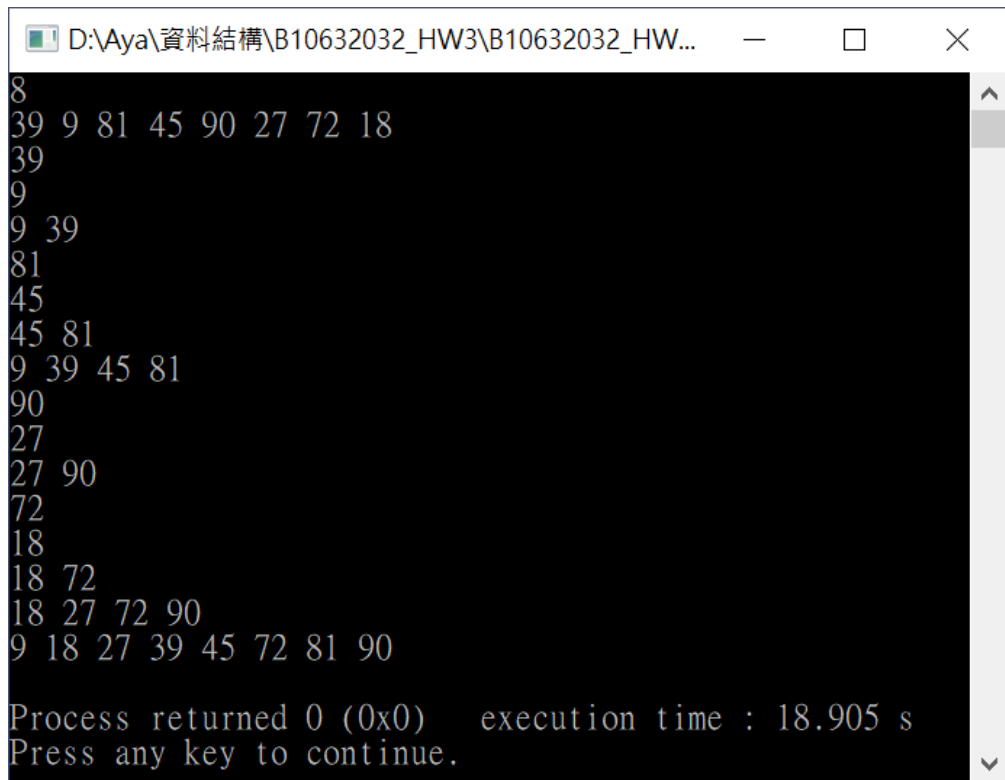
1. Quick sort



```
D:\Aya\資料結構\B10632032_HW3\B10632032_HW3_QS.exe
6
27 10 45 18 25 36
25 10 18 27 45 36
18 10 25 27 45 36
10 18 25 27 45 36
10 18 25 27 36 45

Process returned 0 (0x0)   execution time : 10.292 s
Press any key to continue.
```

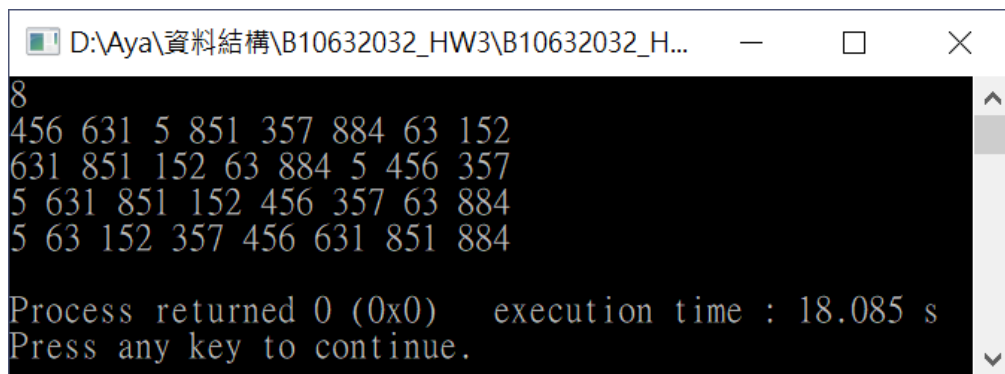
2. Merge sort



```
D:\Aya\資料結構\B10632032_HW3\B10632032_HW...
8
39 9 81 45 90 27 72 18
39
9
9 39
81
45
45 81
9 39 45 81
90
27
27 90
72
18
18 72
18 27 72 90
9 18 27 39 45 72 81 90

Process returned 0 (0x0)   execution time : 18.905 s
Press any key to continue.
```

3. Radix sort



```
D:\Aya\資料結構\B10632032_HW3\B10632032_H...
8
456 631 5 851 357 884 63 152
631 851 152 63 884 5 456 357
5 631 851 152 456 357 63 884
5 63 152 357 456 631 851 884

Process returned 0 (0x0)   execution time : 18.085 s
Press any key to continue.
```

4. Insertion sort

```

D:\Aya\資料結構\B10632032_HW3\B10632032_...
10
39 9 45 63 18 81 108 54 72 36
9 39 45 63 18 81 108 54 72 36
9 39 45 63 18 81 108 54 72 36
9 39 45 63 18 81 108 54 72 36
9 18 39 45 63 81 108 54 72 36
9 18 39 45 63 81 108 54 72 36
9 18 39 45 63 81 108 54 72 36
9 18 39 45 54 63 81 108 72 36
9 18 39 45 54 63 72 81 108 36
9 18 36 39 45 54 63 72 81 108

Process returned 0 (0x0)   execution time : 14.525 s
Press any key to continue.

```

二、速度比較

1. 方式:

將四種排序法放在同一個程式內，將 cout 的部分全部刪掉，並在呼叫排序函式的前後使用 clock()，計算每個函式所花費的時間。其中陣列長度分為 2^{10} 到 2^{17} ；又分成三位數、五位數及七位數三種情形。陣列中的數值使用 rand() 隨機產生。

2. 結果（單位為毫秒）：

三位數								
陣列長度	1024	2048	4096	$\frac{819}{2}$	16384	32768	$\frac{6553}{6}$	131072
QUICK	0	1	1	1	9	15	21	39
MERGE	1	1	1	1	19	25	23	28
RADIX	1	54	97	464	2096	10030	$\frac{4876}{7}$	226124
INSERTION	0	2	2	32	146	589	6041	12127
五位數								
陣列長度	1024	2048	4096	$\frac{819}{2}$	16384	32768	$\frac{6553}{6}$	131072
QUICK	0	0	1	1	4	24	14	42
MERGE	1	2	2	19	10	26	26	54
RADIX	18	35	161	78	3449	16853	$\frac{7254}{0}$	320674
INSERTION	1	1	4	38	202	643	2884	11476
七位數								
陣列長度	1024	2048	4096	$\frac{819}{2}$	16384	32768	$\frac{6553}{6}$	131072
QUICK	0	0	0	0	0	15	15	15
MERGE	0	0	0	0	0	0	16	49
RADIX	15	26	172	774	3319	16474	8075	347394

	3							
INSERTION	0	0	16	39	143	627	2973	12298

3. 心得

從表中可以發現，Radix sort 受到資料量的影響最大，看不出 Merge sort 還是 Quick sort 所受的影響最小，而 Insertion sort 介於中間。而在三位數、五位數跟七位數的表格之間，雖然沒有明顯的差異，但如果只看三位數跟七位數的表格，還是可以看出 Radix sort 會因為位數的增加而花費更多時間。

實際測量的結果和課堂上學到的似乎吻合，覺得很有趣。但其中有一些可能會改變測量結果的因素。例如每個同樣陣列長度及位數的每個排序所排的數值是相同的，但其他陣列長度跟位數所排序的值都是隨機產生，有可能對某個排序法有利，而不利於另一個排序法。所以要減少這種誤差，可能要測試好幾次並平均。此外我的 Radix sort 是用 link list 寫的，如果用二維陣列會花費更多時間；隨著資料量所花費時間增加得更多。

另外我發現在陣列大小 1024 之前，四個排序法所花費的時間幾乎都小於 1 毫秒，所以如果只是排序少量的東西，其實用什麼方法真的沒有很大的差異。